SNAPSHOT · SEPTEMBER 2026 · OPENCLAW 2026.9.x

# OpenClaw Full Stack

The exact toolchain behind a working personal agent: what's installed, what each piece is for, and the order I'd install it in again.

| ~440 | ~327 | ~27K | 7 |
|---|---|---|---|
| SCRIPTS IT WROTE | SKILLS AVAILABLE | LOCAL CALLS IN SEPT | MACHINES, ONE ADMIN |

**Dr Rushindra Sinha (Rushi)**
x.com/irushi

Companion to The OpenClaw Bible

## Contents


01 The core ..... 2
02 Models ..... 3
03 Command-line tools ..... 3
04 Node global packages ..... 4
05 Python ..... 4
06 Memory ..... 4
07 Skills ..... 5
08 Background services ..... 5
09 The fleet ..... 6
10 Fresh install, in order ..... 6


## What this is

A snapshot of the software my agent, Ares, actually runs on, taken from the live Mac mini in September 2026. It pairs with The OpenClaw Bible, which explains why each piece exists; this one is the parts list.

Use it as a shopping list, not a script. Install the core, get one real reply on your phone, then add pieces only when a real task needs them. Every OpenClaw command here was checked against 2026.9.x. Third-party install lines (Homebrew, pip, Ollama) are marked: check each tool's own docs, because package names and model tags move.

> **Install the minimum. Add a tool the second time you need it, not the first time you imagine it.**

**Numbers in this guide** match the Bible: ~10 months of daily use, 94 scheduled automations, 18 specialist agents, ~440 self-written scripts, a knowledge graph of ~6,700 pages and ~28,000 links, 98–100% prompt cache hit on the main agent.

# 01 The core

Five things. Everything else in this guide is optional until a real task says otherwise.

| PIECE | WHY |
|---|---|
| **OpenClaw** 2026.9.x | The agent gateway: channels, sessions, tools, automations, memory |
| **Node.js 26** | OpenClaw's runtime (24.16+ also works) |
| **Claude Code** | The Claude login OpenClaw reuses, and a coding agent in its own right |
| **Tailscale** | Private network between your machines; nothing exposed to the internet |
| **git + gh** | Version the workspace; GitHub from the command line |

```
curl -fsSL https://openclaw.ai/install.sh | bash   # or: npx openclaw@latest
openclaw gateway install                           # run it in the background
openclaw channels status                           # then message it from your phone
brew install gh jq tailscale                       # third-party (Homebrew)
```

## 02 Models

> One subscription for judgement, local models for volume. That split is most of why this doesn't cost a fortune.

| ROLE | WHAT WE RUN |
|---|---|
| Main reasoning | Claude through a subscription login (Sonnet 5 daily, Opus 5.5 for hard problems), thinking high, fast mode off |
| Fallbacks | A second model via `openclaw models fallbacks add <model>`, so an outage doesn't stop the agent |
| Local bulk work | **Ollama** with Qwen 3.5 35B: summarising and extracting from meeting recordings (~27,000 calls in September) |
| Transcription | **mlx-whisper** with `large-v3-turbo` on Apple silicon: every voice note and recording |
| Memory search | Small local embedding models: nomic-embed-text, EmbeddingGemma |

```
brew install ollama              # third-party (Homebrew)
ollama pull <model-tag>          # third-party: pick the tag from the Ollama library; 35B needs a high-RAM Mac
pip install mlx-whisper          # third-party (PyPI), Apple silicon only
openclaw models status           # what's configured, and which auth is live
```

WHAT RUNNING IT TAUGHT ME

- **Local models are free per call and keep private data on the machine**, which is exactly why they sprawl. We once found about 150 GB of models nothing had touched in weeks. Pull what you use.
- **Check how you're billed, don't assume.** A provider name doesn't prove subscription versus per-token billing. `openclaw models status` and `openclaw status --usage` do.

## 03 Command-line tools

> Boring, dependable tools. The agent leans on these more than on anything clever.

| TOOL | JOB |
|---|---|
| `ffmpeg` | Video and audio processing. Check `ffmpeg -buildconf` before relying on filters like text overlays. |
| `pandoc` | Document conversion (Markdown to HTML, PDF, DOCX). This guide is built with it. |
| `uv` | Fast Python environments |
| `himalaya` | Email over IMAP/SMTP (pairs with the bundled `himalaya` skill) |
| `gog` | Google Workspace: Gmail, Calendar, Drive, Sheets, Docs (the bundled `gog` skill sets it up) |
| `ollama` | Local models |

```
brew install ffmpeg pandoc uv himalaya ollama   # third-party (Homebrew)
```

## 04 Node global packages

What's installed globally on the main machine, grouped by job:

| PACKAGE | JOB |
|---|---|
| `openclaw` | The gateway |
| `@anthropic-ai/claude-code` | Claude Code CLI |
| `@openai/codex`, `@google/gemini-cli` | Second-opinion coding agents |
| `mcporter` | Manage MCP servers from the command line |
| `agent-browser` | Headless browser automation |
| `firecrawl-cli` | Reading and searching public web pages |
| `vercel`, `@railway/cli` | Deploying sites and services |
| `docx`, `pptxgenjs`, `sharp` | Generating documents, slides and images |

## 05 Python

Keep one virtual environment for the agent's scripts and always call its Python, never the system one. The agent's own scripts (about 440 of them) should all call it.

```
uv venv ~/.openclaw/workspace/.venv          # third-party (uv)
uv pip install anthropic google-api-python-client google-auth-oauthlib \
  requests beautifulsoup4 pillow pypdf python-pptx playwright
```

WHAT RUNNING IT TAUGHT ME

- **One venv, named in the rules file.** A script that silently picks up the system Python is the classic "it worked yesterday" bug.

## 06 Memory

| LAYER | TOOL |
|---|---|
| Built-in | OpenClaw's Markdown memory: `USER.md`, `MEMORY.md`, daily notes, memory search (`openclaw memory search`) |
| Knowledge graph | **GBrain** (open source, Postgres + pgvector): ~6,700 linked pages on people, projects and decisions, ~28,000 links |
| Verbatim recall | A searchable archive of ~584K past messages, for exact quotes and dates |

WHAT RUNNING IT TAUGHT ME

- **Raise Postgres `max_connections` if many agent sessions share GBrain.** Every session opens its own connection pool. Ours hit the limit until we raised it and closed stale sessions.

## 07 Skills

OpenClaw ships about 50 bundled skills; the agent sees ~327 in total once you add ClawHub, project and personal skills, about 290 of them ready. The ones that earn their keep:

| SKILL | SOURCE | JOB |
|---|---|---|
| `gog` | bundled | Gmail, Calendar, Drive, Sheets, Docs |
| `himalaya` | bundled | Any IMAP/SMTP mailbox |
| `github`, `gh-issues` | bundled | Repos, issues, pull requests |
| `1password` | bundled | Secrets from 1Password |
| `apple-notes`, `apple-reminders` | bundled | Apple's notes and reminders |
| `healthcheck` | bundled | Security and hardening audit of the host |
| `firecrawl-*` | added | Web search and reading public pages |
| `browser-automation` | added | Multi-step browser work |
| `deep-research` | added | Multi-source research with citations |

```
openclaw skills list                    # what's installed and ready
openclaw skills check                   # what's missing requirements
openclaw skills install @owner/<slug>   # from ClawHub
```

Our own public skills: ares-skills. To check the agent can actually find yours: skill-audit-router.

## 08 Background services

> The health system is the part nobody shows. It's also why I trust the rest.

| SERVICE | CADENCE | JOB |
|---|---|---|
| OpenClaw gateway | always | The agent itself (`openclaw gateway install`) |
| Liveness guard | every 2 min | Is the config valid? Is the gateway answering? |
| Job verifier | every 15 min | Did every scheduled job produce the output it declared, with a fresh timestamp? |
| Health report | every 6 h | Full check; alerts only on failure |
| Build health | daily | Every shipped tool runs its own health command |
| Session cleanup | daily | Prunes stale sessions |

**WHAT RUNNING IT TAUGHT ME**

- **Every alert goes through a gate:** dedupe, cooldown, and a repeat only when something gets worse. Before the gate, the same alert fired until I ignored all of them.
- **A green process isn't a working service.** Each check probes the real thing (a reply, a file, a page), not just a port.

## 09 The fleet

| MACHINE | ROLE |
|---|---|
| Mac mini (admin) | The main brain: main agent, memory, automations, local models, fleet admin |
| Second Mac mini | A business agent with its own WhatsApp number |
| iMac | A family-business agent |
| MacBook Pro | Compute and coding |
| Cloud server | A product backend |
| Windows gaming PC | Games and streaming; the agent can drive it for game work |
| Personal laptop | Assist-only: the agent looks and advises, never changes things |

Everything connects over Tailscale and SSH from the admin Mac mini. No ports open to the internet.

## 10 Fresh install, in order

1. Install OpenClaw and finish onboarding (Quick start reuses your Claude login).
2. `openclaw gateway install`, then message the agent from your phone and get a reply.
3. Write `SOUL.md`, `USER.md` and the first rules in `AGENTS.md`. **This is the most important step.**
4. `openclaw config set tools.exec.mode ask`, then `openclaw security audit`.
5. Connect one channel: your own WhatsApp or Telegram chat.
6. Add Google (`gog`) read-only: calendar first, then email.
7. Turn on a morning brief automation (`openclaw automations create`).
8. Put the workspace under git and take a backup (`openclaw backup create`).
9. Only now add local models, more channels, more skills and more machines, one at a time.

**Install the core. Get one real reply.**
**Then earn every addition.**

**More:** docs.openclaw.ai · github.com/rushindrasinha/openclaw-guide · openclaw-mac (one-command Mac setup) · Rushi on X